



J.A.R.V.I.S.

AI OPERATING SYSTEM // MARK XLVII

Comprehensive Technical Specification & Blueprint: Multimodal Conversational AI, Bilingual Speech Synapse, Windows Desktop & Browser Automation, Real-Time Hardware Telemetry, Biometrics & Tactical Glassmorphic Web HUD.

SYSTEM VERSION

v3.2.0 PRODUCTION STABLE

COGNITION PIPELINE

Gemini 2.5/2.0 Flash + Local Knowledge

DEPLOYMENT PORTS

Backend :5000 | Web HUD :8080

STORAGE & AUTH

MongoDB Atlas + OpenCV Biometrics

1. Executive Summary & All 21 Core Capability Pillars

J.A.R.V.I.S. (Just A Rather Very Intelligent System) is an automated full-stack AI OS inspired by the Stark Industries Mark XLVII interface. It bridges multimodal AI cognition, speech synthesis, real-time computer vision, OS automation, and cloud data stores into an integrated tactical HUD.

01

AI Assistant

Context-aware conversational intelligence with persistent memory and Stark persona.

02

Voice Input (STT)

Web Speech & offline Whisper/Vosk recognition in English (en-IN/en-US) & Hindi (hi-IN).

03

Text-to-Speech (TTS)

Multi-engine TTS (WebSpeech, EdgeTTS) with automated Devanagari script routing.

04

LLM / API Cognition

Google Gemini 2.5/2.0 Flash failover with instant offline local knowledge fallback.

05

Desktop Automation

Window controls (min/max/tile), system volume, workstation lock, sleep, reboot, shutdown.

06

File & App Manager

Launches VS Code, Chrome, Spotify, Calculator, plus safe file search & dataset disk upload.

07

Browser Automation

Playwright engine for web navigation, URL normalization, multi-tabs, and DOM parsing.

08

YouTube Controls

Direct video search, automatic playback, volume/fullscreen, and transcript extraction.

09

WhatsApp Messages

Automated messaging via WhatsApp Web & Desktop with contact search and failsafes.

10

Screen Vision & OCR

High-speed screen capture (MSS), window inspection, and Gemini Vision reasoning.

11

Weather Telemetry

Live atmospheric telemetry via Open-Meteo REST service with top header scan & chat cards.

12

Tool Sandbox Execution

Dynamic tool dispatcher, isolated Python execution sandbox, and algorithm builder.

13

Modular Architecture

Clean decoupled structure across frontend, backend, core, actions, and memory layers.

14

Arc Reactor Visualizer

Concentric multi-ring animated core with kinetic audio wave ripple effects.

15

6 Assistant States

Real-time transitions: Standby, Listening, Thinking, Tool Execution, Responding, Success.

16

Web-Based HUD (:8080)

Single-page glassmorphic dashboard with live metrics, side drawer, and telemetry.

17

Biometrics & Auth

OpenCV Haar Cascade face scan, PIN fallback (3000), and 2FA OTP auth terminal.

18

Conversation Logs

Persistent session logs via LocalStorage + SQLite/JSON hybrid store with PDF export.

19

MongoDB Hybrid Data

Cloud MongoDB Atlas integration with automated local fallback to backend JSON stores.

20

Flask REST APIs (:5000)

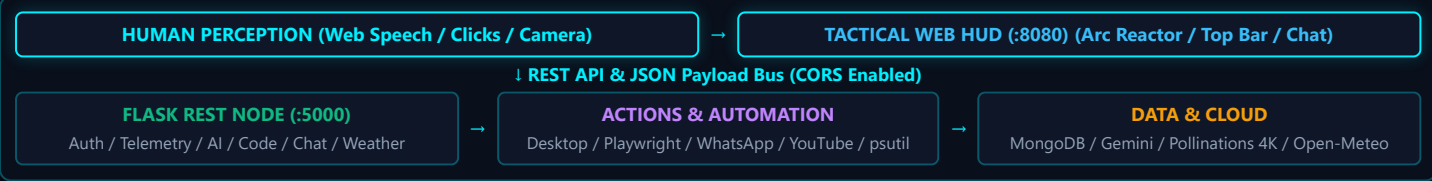
CORS-enabled REST endpoints for system commands, AI, code, auth, chat, and weather.

21

360 Tactical Themes

5 preset tactical themes (Cyan, Green, Red, Gold, Purple) + dynamic 360 color wheel.

2. High-Level Architecture Runtime Pipeline



3. Arc Reactor HUD & Dynamic Assistant States

The central synapse visualizer reacts dynamically across **6 distinct operational states**:

STATE 01

● Standby (Idle)

- Visual Core:

Rhythmic, slow pulsing cyan glow at 0.5Hz.

- System Meaning:

Workstation is armed, listening daemon ready.

- Trigger:

Startup completion or post-response rest.

STATE 02

● Listening

- Visual Core:

Active red/cyan audio wave frequency oscillation.

- System Meaning:

Web Speech API active, receiving en-IN/hi-IN audio.

- Trigger:

Arc Reactor click or floating microphone dock activation.

STATE 03

● Thinking / Synapsing

- Visual Core:

Rapid concentric ring matrix rotation in violet/magenta.

- System Meaning:

Google Gemini reasoning or local knowledge synthesis.

- Trigger:

Query submission (voice or terminal text bar).

STATE 04

● Tool Execution

- Visual Core:

High-velocity orbital pulse with amber warning accents.

- System Meaning:

Running Playwright, WhatsApp send, or Python sandbox.

- Trigger:

Dispatching action via actions/ executor.

STATE 05

● Responding / Vocalizing

- Visual Core:

Kinetic emerald ripple waves synchronized with speech audio.

- System Meaning:

SpeechSynthesisUtterance active; chat bubble streaming.

- Trigger:

TTS playback of assistant answer.

STATE 06

● Success / Confirmation

- Visual Core:

Instantaneous bright emerald flash and chime.

- System Meaning:

Target action completed successfully.

- Trigger:

App opened, volume set, message dispatched, screenshot taken.

4. Web-Based J.A.R.V.I.S. Tactical HUD (Port 8080)

HUD

Glassmorphic Dashboard

Backdrop-filter blur, particle canvas field, digital HUD clock, battery metrics, top header atmospheric scan, and live system log stream.

THEMES

360 Theme Engine

5 tactical presets (Cyan Matrix, Green Diagnostics, Red Combat, Gold Centurion, Purple Quantum) + custom continuous 360 color wheel.

CONTROL

Side Drawer Panel

Slide-out drawer containing user profile, clearance level, cloud database status, project PDF download, and quick navigation shortcuts.

5. Authentication, Biometrics & MongoDB Integration

SECURITY VECTOR	IMPLEMENTATION	ROLE & SPECIFICATION
Optical Facial Scan	OpenCV Haar Cascades (<code>POST /api/auth/face</code>)	Real-time in-memory webcam frame facial detection and biometric user verification.
Passcode Fallback	PIN Entry Mode	4-digit manual security PIN entry with default emergency bypass code (<code>3000</code>).
2FA OTP Verification	Hybrid Auth Flow (<code>POST /api/auth/verify-otp</code>)	6-digit numeric OTP generation & verification for new accounts and credential login.
Tactical Auth Portal	<code>frontend/auth.html</code>	Standalone WebGL GLSL shader portal with interactive pull-cord theme toggle and 5 swatches.
MongoDB Hybrid Data	MongoDB Atlas + JSON/SQLite Fallback	Cloud user profile persistence with local auto-failover to <code>backend/data/users.json</code> .
Conversation History	<code>/api/chat/history</code> + LocalStorage	Full chat log persistence with 1-click PDF export, log search, and history clearance.

6. Native Tool & Automation Subsystems (`actions/`)

01

Browser Automation (Playwright)

File: actions/browser_control.py

- Automated Chrome/Edge headless and headful session management.
- Intelligent URL normalization (e.g. "instagram" to "https://instagram.com").
- DOM interaction: clicking, form filling, scroll management, link scraping.
- Fast fallback to OS default browser via Python webbrowser.

02

YouTube Automation & Transcripts

File: actions/youtube.py

- Direct video playback matching user query terms.
- Video transcript extractor via youtube_transcript_api.
- Controls: play, pause, seek, volume adjustment, fullscreen toggling.

03

WhatsApp & Message Dispatch

File: actions/messaging.py

- Automates WhatsApp Web & desktop client via PyAutoGUI.
- Contact resolution: search contact, open conversation, type and paste message.
- Encrypted email dispatch via SMTP integration.
- PyAutoGUI failsafe protections and automated clipboard management.

04

Screen Processor & Vision

File: actions/screen_processor.py

- Ultra-fast screen capture using mss and PIL.
- Multimodal vision queries routed to Google Gemini Vision.
- Webcam snapshot capture for facial presence verification.

05

Windows Control & App Launcher

Files: actions/open_app.py, desktop.py

- App launcher: VS Code, Chrome, Spotify, Calculator, Notepad, CMD, custom paths.
- Window management: minimize, maximize, restore, close active window.
- System volume control (mute, set percentage), screenshot capture, workstation lock.

06

Code Interpreter & Sandbox

File: actions/code_helper.py

- Automated algorithm generator & LeetCode/Codeforces solver.
- Secure Python subshell execution with 10-second timeout guard.
- Prism.js syntax highlighting with 1-click Copy and Fullscreen Zoom.

7. REST API Endpoints Specification (Port 5000)

CATEGORY	METHOD	ENDPOINT	DESCRIPTION
System	GET	/	Health check & version status report
System	POST	/api/system/command	Hardware diagnostics (CPU/RAM/Temp/Speed) and OS actions
System	POST	/api/system/shell	Execute sandboxed shell commands with 10s timeout
AI	POST	/api/ai/generate	Server-side Google Gemini multi-model routing
Code	POST	/api/code/helper	Code generation, explanation, and sandbox execution
Auth	POST	/api/auth/face	OpenCV Haar Cascade facial biometric verification
Auth	POST	/api/auth/verify-otp	Verify 6-digit OTP and complete authorization
Chat	GET / POST / DEL	/api/chat/history	Retrieve, append, or clear conversation logs
Weather	GET	/api/weather	Live Open-Meteo temperature, atmospheric conditions, and humidity

8. Modular Repository Architecture

JARVIS-AI-OS/	
├── backend/	# Python Flask API Microservices (Port 5000)
│ ├── api/	# REST Blueprints (ai, auth, calls, chat, code, files, system, weather)
│ ├── automation/	# Native OS automation, media playback & hardware telemetry
│ ├── data/	# Local user database records
│ ├── database/	# SQLite/JSON chat history manager
│ ├── config.py	# Backend server configuration
│ └── server.py	# Primary Flask backend server entry point
├── frontend/	# Web HUD & Tactical Interface (Port 8080)
│ ├── index.html	# Main HUD Dashboard (Arc Reactor, Chat Feed, Side Drawer)
│ ├── auth.html	# Standalone Tactical Intelligence Biometric Auth Terminal
│ ├── css/	# Modular CSS (style, dashboard, chat, lockscreen, themes, animations)
│ ├── js/	# Frontend JS (ai, voice, commands, dashboard, settings, utils, chat, weather)
│ └── assets/	# HUD icons, Arc Reactor SVGs, avatar images, sounds
├── core/	# Multimodal Live Engines (STT, TTS, LLM Orchestration)
│ ├── llm.py	# LLM orchestration client (Gemini, Ollama, OpenAI)
│ ├── prompt.txt	# Master system prompt definitions
│ ├── stt.py	# Offline Whisper & Vosk Speech-to-Text
│ └── tts.py	# EdgeTTS, Kokoro, ElevenLabs Text-to-Speech
├── actions/	# Native Agent Tool Handlers
│ ├── browser_control.py	# Playwright browser automation
│ ├── youtube.py	# YouTube streaming & transcript extractor
│ ├── messaging.py	# WhatsApp Web/Desktop & email automation
│ ├── screen_processor.py	# Screen capture OCR & vision analysis
│ ├── open_app.py	# Application & URI launcher
│ ├── computer_control.py	# System control & process supervisor
│ ├── system_monitor.py	# Real-time hardware telemetry (psutil)
│ ├── weather.py	# Weather data provider (Open-Meteo)
│ └── code_helper.py	# Competitive programming sandbox
├── memory/	# Context & Configuration Management (User Profile, Memory Persistence)
├── config/	# Shared Configuration & API key blueprints (api_keys.json.example)
├── docs/	# Comprehensive Technical Documentation (Architecture, APIs, Design)
├── tests/	# Automated Unit & Integration Tests (AI, API, Auth, Backend, Voice)
├── run_dashboard.py	# Unified Full-Stack Launcher (Port 5000 + Port 8080)
├── main.py	# Live Gemini Multimodal Audio Desktop Client
├── ui.py	# PyQt6 Sci-Fi Desktop Client UI
├── requirements.txt	# Python package dependencies
└── package.json	# Project metadata & npm scripts

9. Installation, Quickstart & Launch Guide

STEP 1 Prerequisites Windows 10/11, Python 3.12+ (or 3.10+), modern browser (Chrome, Edge, Brave).	STEP 2 Dependencies pip install -r requirements.txt
STEP 3 Single-Command Full-Stack Launch Starts the Flask backend on Port 5000 and the Tactical Web HUD on Port 8080 simultaneously: py -3.12 run_dashboard.py # Or standard launcher: python run_dashboard.py	
STEP 4 Access Operating System in Browser Open http://127.0.0.1:8080. Enter your Gemini API key in Settings or immediately use the built-in offline fallback engine.	